

AWS Developer Exam: Domain 2 – Implement Encryption Using AWS Services

Overview

Encryption is a foundational concept that must be understood before architecting any AWS workload or application. This task statement focuses on:

- Data protection approaches in AWS

- Encryption at rest and in transit

- AWS key management services

- Certificate management

- Secrets and configuration management

Core Encryption Concepts

Encryption vs. Tokenization

Both are data protection schemes, but they work differently:

Encryption — Transforms data into an unreadable format. The data can only be read again with a secret key.

Tokenization — Replaces sensitive data with a non-sensitive placeholder (token). Adds an additional layer of protection and can help meet compliance requirements (e.g., PCI-DSS).

Two Types of Encryption in AWS

Encryption at rest — Protects data stored on disk, in databases, in S3 buckets, etc.

Encryption in transit — Protects data as it moves between systems, services, or users. Requires secure key and certificate management (e.g., TLS/SSL).

Server-Side vs. Client-Side Encryption

Server-Side Encryption (SSE)

AWS handles encryption and decryption on your behalf as data is written to and read from storage. KMS is commonly used to control when data is decrypted, by whom, and under what conditions.

Client-Side Encryption

Data is encrypted **within your own application** before being sent to AWS. AWS services never see the plaintext data. This removes AWS from your **trust boundary**, making it suitable for highly sensitive workloads or hybrid/on-premises architectures.

Both approaches can use KMS to manage the lifecycle of and permissions on encryption keys, providing a consistent access control mechanism regardless of where data resides (AWS, on-prem, or hybrid).

Amazon S3 Encryption

Important: In S3, **buckets are not encrypted — objects are**. Encryption is defined at the object level.

S3 supports two broad methods of encryption:

Client-Side Encryption

Data is encrypted before upload. S3 stores the already-encrypted object.

Server-Side Encryption (Three Options)

Method	Key Management	Description
SSE-S3	AWS manages keys entirely	Amazon S3-managed keys; simplest option
SSE-KMS	AWS KMS manages keys	You control key policies, rotation, and auditing via KMS
SSE-C	You provide the key with each request	AWS encrypts/decrypts using your supplied key but does not store it

Note: Encryption in transit is standard with S3 and encryption at rest has also recently become a default.

Enforcing Encryption via Bucket Policy

To ensure all objects are encrypted at rest using SSE-KMS, use a **bucket policy** with a condition that denies uploads unless the request includes the `x-amz-server-side-encryption` header.

Amazon EBS Encryption

EBS encryption uses **KMS keys** when creating encrypted volumes and snapshots.

Encryption operations occur on the **servers hosting EC2 instances**, securing both data at rest and data in transit between the instance and its attached EBS storage.

You can attach **both encrypted and unencrypted volumes** to an instance simultaneously.

Understand how EC2 works with KMS to encrypt/decrypt EBS volumes depending on whether the source snapshot is encrypted or unencrypted.

Amazon RDS Encryption

RDS encryption provides an additional layer of data protection. When enabled, the following are encrypted:

The underlying storage for DB instances

Automated backups

Read replicas

Snapshots

Key Exam Questions on RDS Encryption

Can you encrypt a database instance **after** it is created? → **No** — encryption must be set at creation time.

Can you turn off encryption once enabled? → **No** — encryption cannot be disabled once on.

Can you create an encrypted snapshot of an **unencrypted** DB instance? → **No**.

What KMS key do you use to encrypt read replicas in the **same region**? → The **same KMS key** used by the primary.

How do you copy an encrypted snapshot to **another region**? → Specify a KMS key in the destination region when copying the snapshot.

AWS Key Management Service (KMS)

KMS is the central service for creating, managing, and using encryption keys across AWS. For this exam, you need **depth** in KMS and its integration with other services.

Key Principle

Cryptographic keys never leave KMS. Keys are used to perform operations inside KMS but are locked within the service. No one can extract plaintext keys from KMS.

Customer Master Keys (CMKs)

CMKs are the primary resource in KMS used for cryptographic operations. They can be used by you, your applications, or other AWS services. Each CMK stores:

Key ID

Creation date

Resource policy

Description

State (active/inactive)

Key material can be **generated by KMS** or **imported by you**.

Two Types of CMKs

Type	Who Creates It	Rotation	Configurability
AWS Managed CMKs	AWS (e.g., when you use S3 + KMS)	Automatic every 1 year; cannot be disabled	Limited configuration
Customer Managed CMKs	You	Optional; if enabled, rotates every 1 year	Fully configurable — key policies, grants, aliases

Data Encryption Keys (DEKs)

When you need to encrypt large amounts of data, use a **Data Encryption Key (DEK)**:

KMS generates a DEK using a CMK via the `GenerateDataKey` operation.

The DEK is used to encrypt the data locally.

The plaintext DEK is then erased from memory.

Only the **encrypted DEK** is stored alongside the encrypted data.

Envelope Encryption

Envelope encryption is the practice of:

Encrypting plaintext data with a **data key**.

Encrypting the data key with another key (CMK).

Optionally encrypting that key under yet another key.

The top-level plaintext key is the **master key** and must be protected above all else.

KMS Additional Features

Aliases — Create human-readable aliases that point to CMKs.

Multi-Region keys — Share keys or aliases across AWS regions.

Hardware Security Modules (HSMs) — KMS uses HSMs internally to ensure plaintext keys cannot be used outside the HSM by anyone. A key is only used when an authenticated and authorized request is received.

AWS CloudHSM

CloudHSM is used when you need to **directly manage the HSM device** that generates, stores, and uses your encryption keys.

Feature	KMS	CloudHSM
Hardware management	AWS manages	AWS provides hardware; you manage it
Encryption client	AWS handles	You provide your own client
Dedicated hardware	Shared (multi-tenant)	Dedicated (single-tenant)
FIPS compliance	FIPS 140-2 Level 2	FIPS 140-2 Level 3
Tamper resistance	Yes	Yes
Encryption support	Symmetric & Asymmetric	Symmetric & Asymmetric

Best Practice: Add an HSM module in **each Availability Zone** for high availability.

AWS Systems Manager Parameter Store

SSM Parameter Store provides **secure, hierarchical storage** for configuration data and secrets such as passwords, database strings, and license codes.

Key Features

Parameters can be stored as **plaintext** or **encrypted** (using KMS).

Supports **version tracking** of configurations and secrets.

Free to use and highly scalable.

Parameters are organized in a **hierarchy** (e.g., `/myapp/prod/db-password`).

Key API Operations

`GetParameters` — Retrieve one or more parameters by name.

`GetParametersByPath` — Retrieve all parameters under a given path hierarchy.

When to Use Parameter Store

Storing configuration values and non-rotating secrets.

Sharing configuration across multiple Lambda functions or services.

When you need encrypted storage for free with version history.

AWS Secrets Manager

Secrets Manager is purpose-built for storing and managing secrets such as database credentials, API keys, and OAuth tokens.

Key Differences from SSM Parameter Store

Feature	SSM Parameter Store	Secrets Manager
Primary purpose	Configuration & secrets	Secrets only
Automatic rotation	No	Yes (configurable schedule)
Secret generation	No	Yes (auto-generates on rotation)
Cost	Free (standard tier)	Paid per secret
Encryption	Optional (KMS)	Always encrypted with KMS

Exam Scenario: If you need database credentials encrypted **and** automatically rotated on a regular basis (e.g., for RDS in a multi-AZ deployment), use **AWS Secrets Manager**.

AWS Certificate Manager (ACM)

ACM handles the complexity of creating, storing, and renewing public and private **SSL/TLS X.509 certificates** to protect your websites and applications.

Two Options for Managed Certificates

Option	Use Case
ACM (AWS Certificate Manager)	Enterprise customers needing a secure public web presence using TLS
AWS Private CA (formerly ACM PCA)	Enterprise customers building a private PKI inside AWS for internal use only

Additional ACM Capabilities

Issue certificates directly through ACM or **import third-party certificates**.

Export ACM certificates signed by AWS Private CA for use in your **internal PKI** anywhere.

Integrates with services like CloudFront, ALB, API Gateway, and more.

EC2 Key Pairs & Session Manager

EC2 Key Pairs

A key pair (public + private key) is used to prove your identity when connecting to an EC2 instance via SSH.

AWS stores the public key on your instance.

You store the private key locally — anyone with the private key can connect, so it must be secured.

AWS Systems Manager Session Manager (Alternative)

Instead of key pairs, you can use **Session Manager** to connect to EC2 instances:

Interactive browser-based shell — no SSH required.

Works via the AWS CLI.

No need to open inbound ports or manage key pairs.

All session activity can be logged and audited.

Exam Scenario Practice

Scenario 1 — Shared Encrypted Connection String for Lambda + RDS

Requirement: Multiple Lambda functions need to share the same RDS connection string, and it must be encrypted.

Option	Correct?	Why
Lambda environment variable encrypted with KMS	✗	Environment variables are per-function and cannot be shared
Lambda environment variable encrypted with CloudHSM	✗	Lambda uses KMS for encryption, not CloudHSM
IAM execution role with RDS access	✗	Does not encrypt the credentials
SSM Parameter Store (Secure String)	✓	Encrypted with KMS, shared via path, accessible by multiple functions

Scenario 2 — Rotating Database Credentials for RDS Multi-AZ

Requirement: Encrypt and automatically rotate all database credentials, secrets, and API keys.

Answer: AWS Secrets Manager — stores and encrypts credentials with KMS and supports configurable automatic rotation.

Scenario 3 — Encrypting All S3 Objects with SSE-KMS

Requirement: Ensure all objects uploaded to a specific S3 bucket are encrypted at rest using SSE-KMS.

Answer: Apply a **bucket policy** with a condition that denies `PutObject` requests that do not include the `x-amz-server-side-encryption: aws:kms` header.

Key Exam Tips

Buckets are not encrypted — objects are. Encryption is configured at the object level in S3.

KMS keys never leave KMS. All cryptographic operations happen inside the service.

CloudHSM = dedicated hardware + your own client. Use it when you need direct HSM control and FIPS 140-2 Level 3 compliance.

Envelope encryption = encrypt data with a DEK, then encrypt the DEK with a CMK. Use `GenerateDataKey` to implement this locally.

Secrets Manager = secrets + automatic rotation. **SSM Parameter Store** = configuration + optional encryption, no auto-rotation.

RDS encryption must be set at creation — it cannot be added later and cannot be removed.

ACM = public-facing TLS certificates. **AWS Private CA** = internal PKI certificates.

Session Manager is a secure, auditable alternative to EC2 key pairs for instance access.

Lambda functions **cannot share environment variables** — use SSM Parameter Store or Secrets Manager for shared secrets.

Know when to use **SSE-S3 vs. SSE-KMS vs. SSE-C** based on who controls the keys. EOF